

 SOFTWARE ENGINEERING COURSE

LECTURE 1

Introduction to Software Engineering

Micheal Tuape

Adopted from Andrey Saltan

MODULE 1 SCHEDULE

Week	Dates	Topic	SE Chapters
Week 1	September 2 — September 6, 2024	Course Introduction	None
Week 2	September 9 — September 13, 2024	Introduction to Software Engineering	Ch. 1
Week 3	September 16 — September 20, 2024	Software Processes	Ch. 2
Week 4	September 23 — September 28, 2024	Agile Software Development	Ch. 3
Week 5	October 2 — October 8, 2024	Requirements Engineering	Ch. 4
Week 6	October 9 — October 15, 2024	Software Modelling, Coding and Dev.	Ch. 5
Week 7	October 16 — October 22, 2024	Software Architectural Design	Ch. 6

GRADING

»»Course grading structure:

- Weekly Individual Home Assignments/Peer reviews (3:1)– 20%
- Weekly In-class Quizzes – 10%
- Attendance bonus(5%)
- Weekly On-campus Exercises – 10%
- Course Project (reports + presentations) – 25%*
- Guest Lecture 5%
- Final exam – 30%*

(*Minimum 50 points)



AGENDA

»»After this lecture:

- Define software engineering.
- Apricate why software engineering is important.
- Understanding of the challenges of software engineering.
- An understanding of ethical issues around software engineering



SOFTWARE ENGINEERING FAQ

»»What is software?

- Computer programs and associated documentation. Software products that may be developed for a particular customer or may be developed for a general market.



SOFTWARE ENGINEERING FAQ

»»What is software?

Software is a set of instructions, data or programs used to operate computers and execute specific tasks.

What do you think about about this?

- software is also natural, in the sense that it is created by humans at work, with all the attendant constraints and limitations—and thus, like natural language, it is also likely to be repetitive and predictable.

Hindle, Abram, et al. "On the naturalness of software." Communications of the ACM 59.5 (2016): 122-131.



SOFTWARE ENGINEERING FAQ

- Types of software
 - System software
 - Application software
 - Middleware
 - Driver software
 - Programming software (assemblers, compilers, debuggers and interpreters)

SOFTWARE ENGINEERING FAQ

Comparing system software and application software

System software	Application software
General-purpose software that manages basic system resources and processes	Software that performs specific tasks to meet user needs
Written in low-level assembly language or machine code	Written in higher-level languages, such as Python and JavaScript
Must meet specific hardware needs; interacts closely with hardware	Does not take hardware into account and doesn't interact directly with hardware
Installed at the same time as the OS, usually by the manufacturer	User or admin installs software when needed
Runs any time the computer is on	User triggers and stops the program
Works in the background and users don't usually access it	Runs in the foreground and users work directly with the software to perform specific tasks
Runs independently	Needs system software to run



SOFTWARE PRODUCTS

»» Generic products

- Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
- Examples – PC software such as graphics programs, project management tools; C A D software; software for specific markets such as appointments systems for dentists.

»» Customized products

- Software that is commissioned by a specific customer to meet their own needs.
- Examples – embedded control systems, air traffic control software, traffic monitoring systems.



PRODUCT SPECIFICATION

»» Generic products

- The specification of what the software should do is owned by the software developer and decisions on software change are made by the developer.

»» Customized products

- The specification of what the software should do is owned by the customer for the software and they make decisions on software changes that are required.



ESSENTIAL ATTRIBUTES OF GOOD SOFTWARE

»» Maintainability

- Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.

»» Dependability and security

- Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.

»» Efficiency

- Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.

»» Acceptability

- Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.



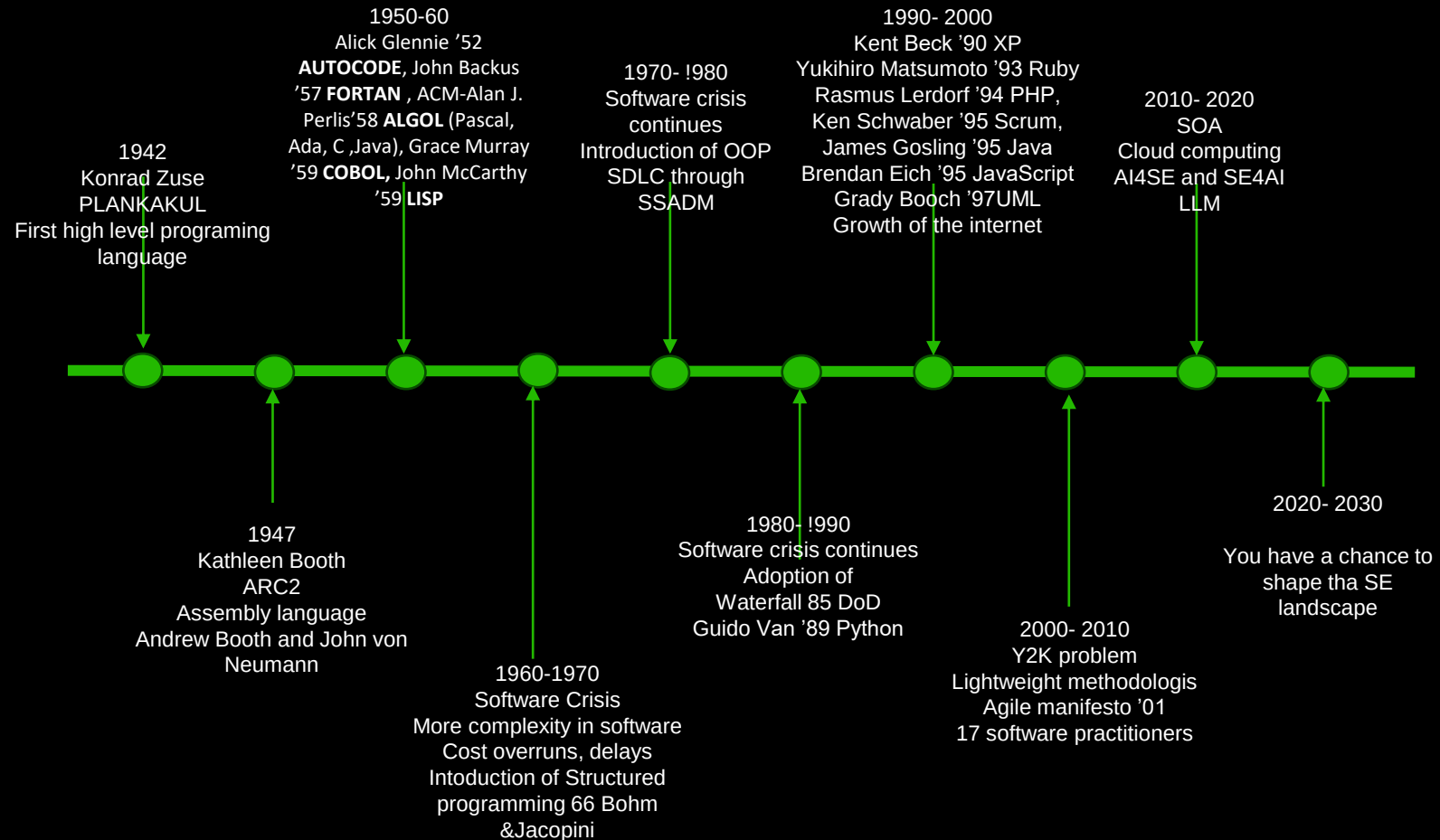
SOFTWARE ENGINEERING

»» The IEEE's 2010 definition states that

software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; that is, the application of engineering to software.

[IEEE 2010]

SOFTWARE ENGINEERING





SOFTWARE ENGINEERING

- »» Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use.
- »» Engineering discipline
 - Using appropriate theories and methods to solve problems bearing in mind organizational and financial constraints.
- »» All aspects of software production
 - Not just technical process of development. Also project management and the development of tools, methods etc. to support software production.



IMPORTANCE OF SOFTWARE ENGINEERING

- More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
- It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project. For most types of system, the majority of costs are the costs of changing the software after it has gone into use.



SOFTWARE PROCESS ACTIVITIES

- » Software specification, where customers and engineers define the software that is to be produced and the constraints on its operation.
- » Software development, where the software is designed and programmed.
- » Software validation, where the software is checked to ensure that it is what the customer requires.
- » Software evolution, where the software is modified to reflect changing customer and market requirements.



GENERAL ISSUES THAT AFFECT SOFTWARE (1 OF 2)

»» Heterogeneity

- Increasingly, systems are required to operate as distributed systems across networks that include different types of computer and mobile devices.

»» Business and social change

- Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software.



GENERAL ISSUES THAT AFFECT SOFTWARE (2 OF 2)

»» Security and trust

- As software is intertwined with all aspects of our lives, it is essential that we can trust that software.

»» Scale

- Software has to be developed across a very wide range of scales, from very small embedded systems in portable or wearable devices through to Internet-scale, cloud-based systems that serve a global community.



SOFTWARE ENGINEERING DIVERSITY

- » There are many different types of software system and there is no universal set of software techniques that is applicable to all of these.
- » The software engineering methods and tools used depend on the type of application being developed, the requirements of the customer and the background of the development team.



APPLICATION TYPES

»» Stand-alone applications

- These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.

»» Interactive transaction-based applications

- Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.

»» Embedded control systems

- These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.



APPLICATION TYPES

»» Batch processing systems

- These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.

»» Entertainment systems

- These are systems that are primarily for personal use and which are intended to entertain the user.

»» Systems for modeling and simulation

- These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects.



APPLICATION TYPES (3 OF 3)

»» Data collection systems

- These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.

»» Systems of systems

- These are systems that are composed of a number of other software systems.



SOFTWARE ENGINEERING ETHICS

- » Software engineering involves wider responsibilities than simply the application of technical skills.
- » Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- » Ethical behaviour is more than simply upholding the law but involves following a set of principles that are morally correct.



ISSUES OF PROFESSIONAL RESPONSIBILITY (1 OF 2)

»» Confidentiality

- Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

»» Competence

- Engineers should not misrepresent their level of competence. They should not knowingly accept work which is not within their competence.



ISSUES OF PROFESSIONAL RESPONSIBILITY (2 OF 2)

»» Intellectual property rights

- Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.

»» Computer misuse

- Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).



ACM/IEEE CODE OF ETHICS

- »» The professional societies in the US have cooperated to produce a code of ethical practice.
- »» Members of these organisations sign up to the code of practice when they join.
- »» The Code contains eight Principles related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of the profession.



RATIONALE FOR THE CODE OF ETHICS

- Computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society at large. Software engineers are those who contribute by direct participation or by teaching, to the analysis, specification, design, development, certification, maintenance and testing of software systems.
- Because of their roles in developing software systems, software engineers have significant opportunities to do good or cause harm, to enable others to do good or cause harm, or to influence others to do good or cause harm. To ensure, as much as possible, that their efforts will be used for good, software engineers must commit themselves to making software engineering a beneficial and respected profession.



THE ACM/IEEE CODE OF ETHICS (1 OF 2)

Software Engineering Code of Ethics and Professional Practice

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

Preamble

The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.



THE ACM/IEEE CODE OF ETHICS (2 OF 2)

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:



ETHICAL PRINCIPLES (1 OF 2)

1. Public - Software engineers shall act consistently with the public interest.
2. Client and Employer - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. Product - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. Judgment - Software engineers shall maintain integrity and independence in their professional judgment.



ETHICAL PRINCIPLES (2 OF 2)

5. Management - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. Profession - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. Colleagues - Software engineers shall be fair to and supportive of their colleagues.
8. Self - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.



SOFTWARE ENGINEERING FUNDAMENTALS

- »» Some fundamental principles apply to all types of software system, irrespective of the development techniques used:
 - Systems should be developed using a managed and understood development process. Of course, different processes are used for different types of software.
 - Dependability and performance are important for all types of system.
 - Understanding and managing the software specification and requirements (what the software should do) are important.
 - Where appropriate, you should reuse software that has already been developed rather than write new software.



KEY POINTS (1 OF 2)

- » Software engineering is an engineering discipline that is concerned with all aspects of software production.
- » Essential software product attributes are maintainability, dependability and security, efficiency and acceptability.
- » The high-level activities of specification, development, validation and evolution are part of all software processes.
- » The fundamental notions of software engineering are universally applicable to all types of system development.



KEY POINTS (2 OF 2)

- » There are many different types of system and each requires appropriate software engineering tools and techniques for their development.
- » The fundamental ideas of software engineering are applicable to all types of software system.
- » Software engineers have responsibilities to the engineering profession and society. They should not simply be concerned with technical issues.
- » Professional societies publish codes of conduct which set out the standards of behaviour expected of their members.

